Relatório de Melhoria – Dashboard Meteorológico

SENAI São Paulo

Desafio de Melhoria 05.1

Aluna: Sophia

Projeto: OpenWeatherMap + MQTT + JavaScript

Relatório de melhoria do projeto existente

1. Introdução

Este trabalho foi feito a partir de um projeto que já consultava o clima usando a API OpenWeatherMap. O sistema também utilizava JavaScript, MQTT e um gráfico para mostrar temperatura e umidade.

O objetivo foi melhorar a aparência e a facilidade de uso da página, acrescentando informações e funções para deixar o dashboard mais completo.

2. Qual era o problema da versão original?

A versão original era mais simples, com poucas informações meteorológicas e um visual básico. A temperatura e a umidade apareciam sem uma organização visual tão completa, e o gráfico também precisava de melhorias na apresentação.

Também era importante acrescentar recursos que ajudassem o usuário a consultar o clima, entender os resultados e acompanhar as consultas feitas.

3. Quais melhorias foram realizadas?

Foi preparado um novo visual para o dashboard, com tema escuro, cartões de informações e organização melhor das áreas da página.

A interface apresentada nos prints possui consulta por cidade e país, botão para buscar o clima, opção para salvar e usar uma cidade favorita, indicação de última atualização e status MQTT conectado.

O gráfico tem título, legenda e identificação dos eixos, mantendo a visualização de temperatura e umidade ao longo das consultas. Também aparece uma lista de consultas recentes.

4. Quais novas informações da API foram utilizadas?

Além da temperatura e da umidade, o dashboard apresenta três novas informações retornadas pela OpenWeatherMap: sensação térmica, velocidade do vento e pressão atmosférica.

A sensação térmica é apresentada em graus Celsius, a velocidade do vento em metros por segundo e a pressão atmosférica em hPa.

5. Qual foi a nova funcionalidade criada?

Uma funcionalidade adicional é a possibilidade de salvar uma cidade favorita e usar essa cidade novamente. A interface também mantém um histórico de consultas recentes, com cidade, temperatura, umidade e horário.

6. Qual foi a maior dificuldade encontrada?

Uma das dificuldades foi atualizar o código sem perder o funcionamento que já existia. Como o projeto usa HTML, CSS e JavaScript, foi necessário tomar cuidado para manter a consulta da API e a comunicação MQTT funcionando.

Também é importante configurar corretamente a chave da OpenWeatherMap e testar as consultas com a internet funcionando.

7. Como o problema foi resolvido?

As mudanças foram organizadas no arquivo HTML do projeto, mantendo a OpenWeatherMap como API meteorológica e preservando a integração MQTT. O código foi ajustado para organizar melhor as informações e apresentar mais dados do clima.

Para usar o projeto, é necessário colocar uma chave válida da OpenWeatherMap no local indicado no código. A chave real não deve aparecer em capturas de tela nem ser publicada em um repositório público.

8. O que aprendi sobre APIs?

Aprendi que uma API permite que um programa peça informações a outro serviço. Neste projeto, o JavaScript faz uma requisição para a OpenWeatherMap e recebe os dados meteorológicos em formato JSON.

Depois, o código lê os campos necessários desse JSON e mostra as informações na página.

9. O que aprendi sobre MQTT?

Aprendi que o MQTT permite enviar mensagens usando tópicos. O broker funciona como intermediário: recebe as mensagens publicadas e as encaminha para os clientes inscritos nos tópicos correspondentes.

No projeto, a comunicação MQTT é usada para publicar informações como temperatura, umidade e cidade. No print, o status aparece como conectado.

10. O que aprendi sobre JavaScript?

Aprendi que JavaScript pode buscar dados na internet, ler respostas em JSON, atualizar elementos HTML, responder a cliques e teclas do usuário e atualizar gráficos.

Também percebi a importância de tratar erros e mostrar mensagens claras para que a pessoa saiba quando a consulta foi concluída ou quando algo não deu certo.

11. Testes e evidências

Os prints a seguir mostram a interface do dashboard com uma consulta meteorológica, os cartões com informações, o gráfico e a lista de consultas recentes. A tela mostra o clima de Rio de Janeiro, a última atualização e o status MQTT conectado.

Antes da entrega final, também é recomendado testar o campo vazio, uma cidade inexistente, a cidade favorita, a troca de tema, a visualização em tela estreita e o comportamento quando a internet não estiver disponível.



Figura 1 – Tela principal com os cartões meteorológicos e a consulta de cidade.

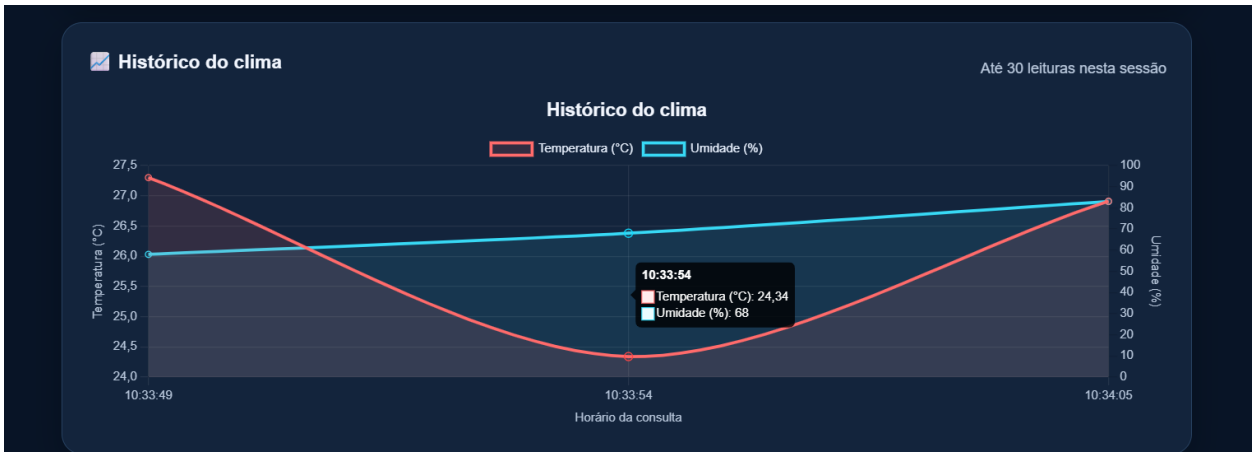


Figura 2 – Gráfico do histórico de temperatura e umidade.

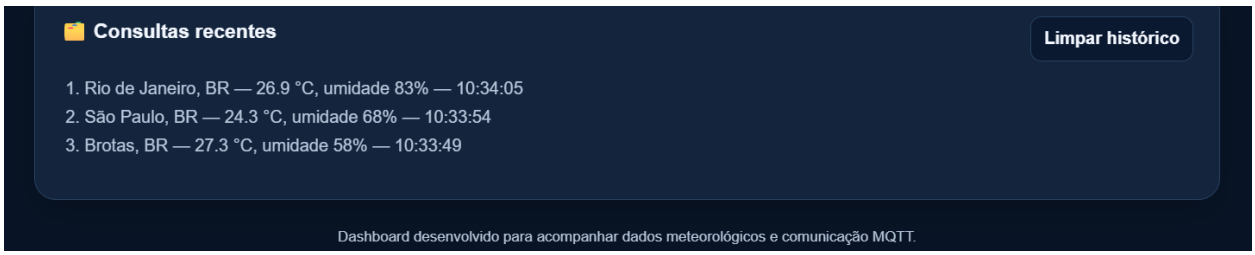


Figura 3 – Lista de consultas recentes.

12. Se tivesse mais tempo, o que acrescentaria?

Se tivesse mais tempo, eu acrescentaria alertas para condições climáticas importantes, como temperaturas muito altas ou vento forte. Também seria interessante comparar duas cidades lado a lado para ajudar o usuário a acompanhar locais diferentes.

13. Conclusão

Com esta melhoria, o projeto fica mais organizado e oferece mais informações para acompanhar o clima. A atividade ajudou a praticar HTML, CSS e JavaScript, além de entender melhor como uma API e o MQTT podem ser usados juntos em uma aplicação.

Como próximo passo, é importante executar os testes finais e confirmar que todos os recursos estão funcionando no navegador.